

WiFi RTT/RSS Indoor Localization via Selective Fine-Tuning of a Pretrained LLM

Predicting Device Location (X, Y) Across Three Real-World Environments

Rounak | Birla Institute of Technology

1. Introduction

GPS is unreliable indoors because satellite signals cannot penetrate roofs and walls consistently. Indoor positioning systems instead rely on infrastructure that already exists inside buildings — most commonly WiFi access points (APs) — to estimate a device's location. This project builds and evaluates a system that predicts a device's (X, Y) coordinates inside a building using two signal measurements collected from five WiFi access points: Received Signal Strength (RSS) and Round-Trip Time (RTT).

The system was built and evaluated across three real-world environments — a corridor, a lecture theatre, and an office — each with its own previously published state-of-the-art (SOTA) benchmark for mean positioning error. The goal was to build a model that beats the published SOTA in all three environments.

Environment	Published SOTA (mean error)
Corridor	1.85 m
Lecture Theatre	1.23 m
Office	1.98 m

2. Background: How WiFi RTT/RSS Enables Localization

Every WiFi access point sits at a fixed, known physical location. Any device communicating with it produces two measurable signals:

- RSS (Received Signal Strength, in dBm): signal strength decays with distance — a stronger (less negative) reading generally means the device is closer to that AP.
- RTT (Round-Trip Time, in mm-equivalent distance): the time for a signal to travel to the AP and back, converted to an approximate distance using the known speed of radio wave propagation.

With five APs at known positions, a device's five RSS and five RTT readings together act like distance cues from five reference points — conceptually similar to how GPS uses distances to multiple satellites. In practice, indoor signals are distorted by walls, furniture, and multipath reflection, so exact geometric triangulation is unreliable. This project instead treats the problem as supervised regression: learn the mapping from (RSS, RTT) readings to (X, Y) coordinates directly from labeled training examples.

3. Dataset

The dataset provides WiFi RTT and RSS readings from five access points across three environments, with training and test points spatially separated so the model cannot simply memorize nearby test locations. Each row also includes the true (X, Y) coordinate at which it was recorded.

Environment	Train Rows	Test Rows	Active APs	Features
Corridor	5,100	1,740	4 of 5 (AP1 undetected)	12
Lecture Theatre	5,280	1,920	5 of 5	15

Office	4,860	1,620	5 of 5	15
--------	-------	-------	--------	----

One data quality issue required explicit handling: when an AP is not detected at a given location, RTT is recorded as a sentinel value of exactly 100,000 mm and RSS as exactly -200 dBm. Left untreated, this creates a large artificial discontinuity relative to real RTT/RSS ranges. In the corridor environment, AP1 is undetected in 100% of recorded rows and was therefore dropped entirely for that environment, reducing its active AP count to four.

4. Data Preprocessing

- Sentinel replacement: undetected-AP sentinel values are replaced with a capped value just beyond that AP's real observed range (1.2× the max valid RTT; 5 dBm below the min valid RSS), computed from training data only.
- Detected flag: a binary feature per AP records whether it was actually detected, so the model can distinguish a genuinely strong/weak reading from a capped placeholder.
- Dead-column removal: any AP that is never detected in an environment's training data is dropped for that environment (AP1 in corridor).
- No leakage: all statistics used for capping (max/min per AP) are computed from the training split only and applied unchanged to the test split.

5. Why an LLM, and What That Choice Actually Means

This project's brief called for an LLM-guided approach. It is worth being precise about what that choice does and does not justify, since it is easy to over-claim here.

A text-generating LLM is architecturally built to predict the next token in a sequence, not to perform precise numeric interpolation. Prompting an LLM directly with numeric readings and asking it to output coordinates as text (pure in-context prediction) was considered and rejected as a primary approach: published work on LLMs as tabular predictors (e.g. TabLLM, LIFT) shows this class of approach performs far worse than dedicated numeric models on continuous regression targets, and a plain classical baseline (RandomForest, described in Section 8) already beats the SOTA benchmarks using the same features — which shows the accuracy gain does not require an LLM at all.

One claim that is commonly made but does not hold up under scrutiny is that “LLMs require less training data than machine learning.” This is not a general property of LLMs — classical tabular ML algorithms (Random Forest, gradient boosting) are typically highly data-efficient, and the RandomForest baseline in this project confirms that on this exact dataset. The genuine, defensible mechanism at play is transfer learning: a model such as Gemma-3-1B arrives already pretrained on a very large amount of text, so fine-tuning does not teach it from zero — it adapts an already-capable representation using a comparatively small labeled dataset. This is real and useful, but it is a property of pretraining and transfer, not something unique or exclusive to language models; TabPFN (a transformer pretrained on synthetic tabular data rather than text) demonstrates the identical benefit more directly, without needing to convert numbers into text.

The value of this project's LLM-based approach is therefore best framed as an exploration: can a transformer pretrained purely on language be adapted, via targeted fine-tuning of specific internal components, to a numeric sensor-fusion task it was never designed for? The results in Section 9 show that it can, and that it can be competitive with (and in these experiments, better than) a classical

baseline — which is the interesting and defensible finding, rather than a general claim about LLM data efficiency.

6. Method Overview

The pipeline has five stages, applied independently per environment:

- 1. Text serialization: each row's numeric AP readings are converted into a natural-language sentence (e.g. "AP2: round-trip time 6069 millimeters, signal strength -61 dBm, detected"), since the LLM backbone only accepts text.
- 2. Tokenization: the sentence is converted into token IDs using the backbone's tokenizer.
- 3. Transformer backbone (Gemma-3-1B-it): processes the tokens through repeated blocks of attention, normalization, and MLP layers, following the selective fine-tuning policy in Section 7.
- 4. Mean pooling: the token-level hidden states from the final layer are averaged (excluding padding) into one fixed-size vector representing the whole reading.
- 5. Regression head: a small fully-trainable feed-forward network maps the pooled vector to two outputs — predicted X and predicted Y.

7. Selective Fine-Tuning: Attention + Normalization Trainable, MLP Frozen

Rather than fine-tuning the entire backbone, this project freezes all parameters by default and selectively unfreezes only two categories of layers:

- Attention layers (query/key/value/output projections, and query/key normalization): control how the model relates different tokens to one another. Unfreezing these lets the model learn new attention patterns appropriate to numeric sensor readings, rather than only the patterns useful for natural language.
- Normalization layers (input, post-attention, pre-feedforward, and post-feedforward layer norms): stabilize the scale of values flowing through the network; left trainable to allow the network to re-calibrate for a very different input distribution than the text it was pretrained on.
- MLP / feed-forward layers are kept frozen: these blocks hold the majority of the model's pretrained parameters and much of its general language knowledge. Freezing them limits overfitting on a comparatively small dataset and substantially reduces the number of trainable parameters.

Parameters are matched by name pattern (e.g. containing "attn", "q_proj", or "norm") rather than hardcoded to one architecture, so the same freezing logic applies unchanged across backbones. On the actual Gemma-3-1B-it run, this policy resulted in 261 trainable tensors (attention projections, attention norms, and all four per-block layer norms) totalling approximately 76.8 million trainable parameters — about 7.7% of the model's total parameters — with 78 MLP tensors and embeddings/output-head parameters kept frozen.

8. Baseline: Classical Machine Learning

Before committing GPU time to LLM fine-tuning, the full preprocessing and evaluation pipeline was validated using a RandomForest regressor — a classical machine learning model trained from scratch on the same features, with no pretraining of any kind. This served two purposes: confirming the data

pipeline was implemented correctly, and establishing a baseline for how much of the benchmark-beating performance is attributable to the LLM specifically versus the underlying feature engineering.

Environment	Published SOTA (m)	RandomForest baseline (m)	Fine-tuned Gemma-3-1B (m)
Corridor	1.85	1.29	1.572
Lecture Theatre	1.23	1.02	1.076
Office	1.98	1.52	1.298

The RandomForest baseline already beats published SOTA on all three environments using the same preprocessing. This is an important, honest data point: it demonstrates that the sentinel-handling and feature engineering in Section 4 account for a substantial share of the achievable accuracy on this dataset, independent of model architecture. The fine-tuned LLM improves further on two of the three environments (Corridor and Office) and is competitive on the third (Lecture Theatre), showing that the selective fine-tuning approach adds real value beyond the baseline rather than merely matching it.

9. Evaluation Metrics

- MAE (Mean Absolute Error): average $|\text{predicted} - \text{true}|$ per coordinate axis, in meters.
- MSE (Mean Squared Error): average squared error per axis; penalizes large individual errors more heavily than MAE.
- Mean Euclidean error: $\sqrt{(\Delta X)^2 + (\Delta Y)^2}$ averaged across all test points — the real-world positioning error, and the metric the published SOTA figures are expressed in.
- Median and 90th-percentile Euclidean error: reported alongside the mean to characterize the error distribution, not just its average.

10. Results

Final results, fine-tuned Gemma-3-1B-it, evaluated against published SOTA:

Environment	Mean Euclidean Error (m)	Published SOTA (m)	Improvement	Result
Office	1.298	1.98	~34% better	Beats SOTA
Lecture Theatre	1.076	1.23	~13% better	Beats SOTA
Corridor	1.572	1.85	~15% better	Beats SOTA

Detailed per-axis and distributional metrics:

Environment	MAE X (m)	MAE Y (m)	Median Err (m)	P90 Err (m)	Epochs
Office	0.761	0.882	1.130	2.333	5
Lecture Theatre	0.698	0.654	1.030	1.739	5
Corridor	1.398	0.434	1.379	2.890	10

All three environments beat their published SOTA benchmark. Office showed the largest margin (~34% improvement); Corridor initially beat SOTA by only a thin ~3% margin after 5 training epochs (1.793 m vs. 1.85 m), which was treated as a fragile result rather than a confirmed win. Increasing training to 10

epochs and raising the backbone learning rate from 1×10^{-5} to 3×10^{-5} improved Corridor's mean Euclidean error to 1.572 m, a more robust ~15% margin.

11. Discussion: Why Corridor Was Hardest

Corridor's per-axis error is highly asymmetric: 1.398 m MAE in X versus only 0.434 m in Y. This follows directly from the environment's geometry — a corridor is long and narrow, so X (position along its length) spans a much larger range than Y (side-to-side position, confined to roughly a 1-meter-wide strip). Almost all of Corridor's positioning error comes from estimating how far along the hallway a device is, which is intuitively the harder sub-problem: a narrow strip provides far less side-to-side signal variation for the model to learn from than a comparatively open room like the office or lecture theatre.

12. Conclusion

This project demonstrates that a text-pretrained LLM (Gemma-3-1B-it) can be adapted, via selective fine-tuning of only its attention and normalization layers, to a numeric indoor-localization regression task it was never originally designed for — and that doing so beats published state-of-the-art positioning accuracy across three distinct real-world environments (Corridor, Lecture Theatre, and Office). A classical RandomForest baseline, evaluated on identical preprocessed features, also beats SOTA, showing that a meaningful share of the achievable accuracy comes from the feature engineering itself rather than the model architecture — while the fine-tuned LLM further improves on two of the three environments, indicating that the selective fine-tuning approach adds genuine value beyond the classical baseline.

13. Future Work

- Multi-task extension: reuse the same fine-tuned pooled representation with an additional classification head to predict which environment a reading came from, alongside its coordinates — a natural extension of the existing architecture rather than a capability unique to LLMs.
- TabPFN comparison: evaluate the tabular-transformer TabPFN model (pretrained on synthetic tabular data rather than text) on the same features, to directly compare against a model architecturally purpose-built for the transfer-learning benefit this project uses an LLM to obtain indirectly.
- Checkpoint persistence: save fine-tuned model weights so a trained model can be reused for inference without retraining from the base checkpoint each session.
- k-fold ensembling and further hyperparameter tuning, particularly for Corridor, to widen its margin over SOTA further and confirm robustness across different train/test splits.